DEEPGROUNDREPAIR: A Tool for API-Grounded Generation and Shape-Aware Debugging of Deep Learning Programs

Xiangyue Ma[1], Zhen Wang[1], Jun Li[1], Haotian Li[1], Chenglong Li[2], Xiaoting Du[3], and Zheng Zheng[1]
[1]Beihang University, Beijing, China
Email: {xiangy_ma, wz2603520, 2503515, lihaotian08, zhengz}@buaa.edu.cn
[2]Beijing University of Technology, Beijing, China
Email: lichenglong@bjut.edu.cn
[3]Beijing University of Posts and Telecommunications, Beijing, China
Email: duxiaoting@bupt.edu.cn

***Abstract*—Large language models (LLMs) can generate deep learning (DL) programs from natural-language requirements, but the generated programs may contain API-level syntactic issues or fail at runtime because of tensor-shape mismatches. We present DEEPGROUNDREPAIR, an executable tool that integrates API-grounded generation, root-cause localization, and shape-aware repair into a unified workflow. It grounds draft code generation in task-relevant API knowledge, localizes upstream API configurations behind tensor-shape failures, and uses runtime tensor shapes to guide parameter repair. Experimental results on 100 model-level DL programming tasks show that the tool improves code executability and reduces API-level syntactic issues compared with four baselines. DEEPGROUNDREPAIR supports TensorFlow, PyTorch, and PaddlePaddle and is intended for DL developers and software engineering researchers working with LLM-generated DL code. The artifact and demonstration video are publicly available at `https://github.com/maxy-635/DeepGroundRepair` and `https://youtu.be/pq7gUaa3C3s`, respectively.**

***Index Terms*—deep learning programs, large language models, code generation, root-cause localization, program repair**

I. INTRODUCTION

Large language models (LLMs) offer a promising way to generate deep learning (DL) programs from natural-language requirements. However, generating executable DL code remains challenging due to library-specific API constraints and complex tensor-shape dependencies. LLM-generated programs may invoke nonexistent or misused APIs, or fail at runtime due to incompatible tensor shapes [1], [2]. Such failures are particularly difficult to resolve because the operation reporting a mismatch may be far downstream from the API configuration responsible for it. Correcting the reported operation alone may therefore leave the underlying problem unresolved.

Existing retrieval-augmented generation (RAG) and execution-feedback-guided repair methods only partially address these challenges. RAG methods struggle to retrieve relevant APIs due to the gap between high-level task requirements and fine-grained API knowledge [3]–[6]. Meanwhile, traceback-guided repair identifies where execution fails but provides limited guidance for tracing tensor-shape mismatches to upstream configurations. This limitation is particularly significant because tensor-shape mismatches account for 77.34% of runtime failures on average in our prior study [7]. These challenges highlight the need for a DL-specific solution that connects verified API knowledge with root-cause localization and explicit tensor-shape constraints [8].

These challenges motivate DEEPGROUNDREPAIR, whose retrieval, localization, and repair algorithms are detailed in an associated research study available on arXiv [9]. Building on this study, we develop an executable tool that integrates these algorithms into a unified *Generate* → *Localize* → *Repair* workflow.

This paper focuses on the tool-level implementation and practical usage of DEEPGROUNDREPAIR. The tool provides stage-specific execution scripts, configurable LLMs, and library-specific API knowledge bases. It exposes intermediate artifacts, including retrieved API knowledge, generated code, localized root causes, tensor-shape annotations, and repaired programs. These artifacts allow users to inspect the localization and repair process and validate the repaired programs through execution. A representative demonstration illustrates these capabilities using a tensor-shape mismatch.

II. DEEPGROUNDREPAIR TOOL DESIGN

Figure 1 presents the architecture of DEEPGROUNDREPAIR. Its three stages form an executable pipeline supported by locally deployed LLMs and library-specific API knowledge bases.

API-Grounded Generation. Given a natural-language task requirement and a target DL library, DEEPGROUNDREPAIR recommends task-relevant APIs, verifies their existence, and retrieves corresponding API documentation from the knowledge base. The retrieved knowledge is then provided to the LLM to generate a draft DL program. This stage produces the initial code for subsequent execution and localization [4], [5].

Root-Cause Localization. The tool executes the generated program and captures runtime failures. For tensor-shape mis-

matches, it combines traceback information with static and dynamic analysis to trace the failure to its upstream API configuration. The localization identifies the root-cause API, suspicious parameters, and relevant tensor shapes. It then masks the suspicious parameters in the generated code, producing parameter-masked code for subsequent repair.

Shape-Aware Repair. The tool annotates the parameter-masked code with traced tensor shapes and retrieves relevant shape-related API knowledge. The annotated code and retrieved knowledge are provided to the LLM to guide targeted repair of the identified parameters. The resulting program is re-executed to validate whether the runtime failure has been resolved. For non-shape-related runtime failures, the tool applies a single-round traceback-guided repair strategy.

III. Tool Usage

A. Supporting Resources

Implementation and Installation. DeepGroundRepair is implemented in Python and supports TensorFlow, PyTorch, and PaddlePaddle. Our experiments were conducted on a server equipped with two NVIDIA V100 GPUs, running CentOS Linux 7.8, Python 3.10, CUDA 12.1, and cuDNN 8.9.5. The DL library versions were TensorFlow 2.16.1, PyTorch 2.11.0, and PaddlePaddle 3.3.0. The artifact provides a Conda environment specification (`environment.yaml`) for installing the required dependencies, with detailed installation instructions available in the artifact repository.

LLM Deployment. DeepGroundRepair supports locally deployed open-weight LLMs through Hugging Face Transformers. The current implementation uses models from the Gemma-3 and Mistral families, covering parameter sizes from 3B to 27B. Users can configure the local model checkpoints in the provided execution scripts to perform API recommendation, code generation, and shape-aware repair.

DL API Knowledge Base. DeepGroundRepair incorporates library-specific API knowledge bases constructed from the official documentation of TensorFlow, PyTorch, and PaddlePaddle, containing 3,686, 4,137, and 1,531 API entries, respectively. Each knowledge base provides sparse and dense retrieval indexes to support Stages 1 and 3 [10], [11]. Specifically, Stage 1 retrieves API names, signatures, and descriptions for draft code generation. Stage 3 additionally retrieves parameter descriptions and available tensor-shape transformation formulas to guide shape-aware repair.

B. Tool Configuration and Execution

Input and Configuration. The current prototype uses natural-language task requirements from the DeepEval benchmark [7], stored as YAML files under `benchmark/DeepEval/`, each containing a `Requirement` field. Before execution, users configure the target DL library (`--dlls`), benchmark (`--benchmark`), LLM checkpoint, experiment ID, and number of generation repeats (`--repeats`) in the provided scripts. Consistent model and experiment IDs are maintained across the three stages to ensure correct access to intermediate results. The repository also provides preconstructed API knowledge bases under `database/`.

Tool Execution. After configuration, users invoke the three stages sequentially through their respective `run_main.sh` scripts under `run_bashes/step_one/`, `run_bashes/step_two/`, and `run_bashes/step_three/`. Stage 1 generates draft programs (`generated_code.py`) and API retrieval records. Stage 2 diagnoses execution failures and produces `annotated_code.py` and `annotated_code.json`, containing parameter-masked code and structured diagnostic information. Stage 3 uses these artifacts to generate `repaired_code.py` and validates the repair through re-execution.

The generated artifacts are stored under `response/`, with execution logs under `logs/`. Users can inspect the masked parameters and localization results, examine the associated tensor-shape evidence, and compare the draft and repaired programs. Execution logs indicate whether the repaired program passes validation. The repository also provides evaluation scripts for code executability, API-level syntactic issues, and semantic consistency.

TABLE I
API-level syntactic issues per 10 programs, averaged across six LLMs.

Method	TensorFlow	PyTorch	PaddlePaddle
B1	5.82	3.14	27.35
B2	4.44	1.36	11.09
B3	5.38	1.86	22.42
B4	4.28	**1.08**	9.88
DeepGR	**0.94**	1.30	**6.08**

TABLE II
Code Executability Rate (CER, %) averaged across six LLMs.

Method	TensorFlow	PyTorch	PaddlePaddle
B1	37.8	17.0	12.2
B2	36.7	22.7	8.3
B3	51.2	26.5	19.5
B4	49.0	31.3	17.8
DeepGR	**58.2**	**41.2**	**31.5**

IV. Evaluation and Representative Demonstration

A. Evaluation

The associated research study evaluates DeepGroundRepair on 100 model-level DL programming tasks using six open-weight LLMs across TensorFlow, PyTorch, and PaddlePaddle [9]. Four baselines are considered. *Vanilla LLM* (B1) generates code directly from the task requirement; *Hybrid-RAG* (B2) augments generation with API knowledge retrieved using sparse and dense methods [10]. B3 and B4 extend B1 and B2, respectively, with traceback-guided self-debugging, in which the LLM revises a failed program using execution error feedback [12].

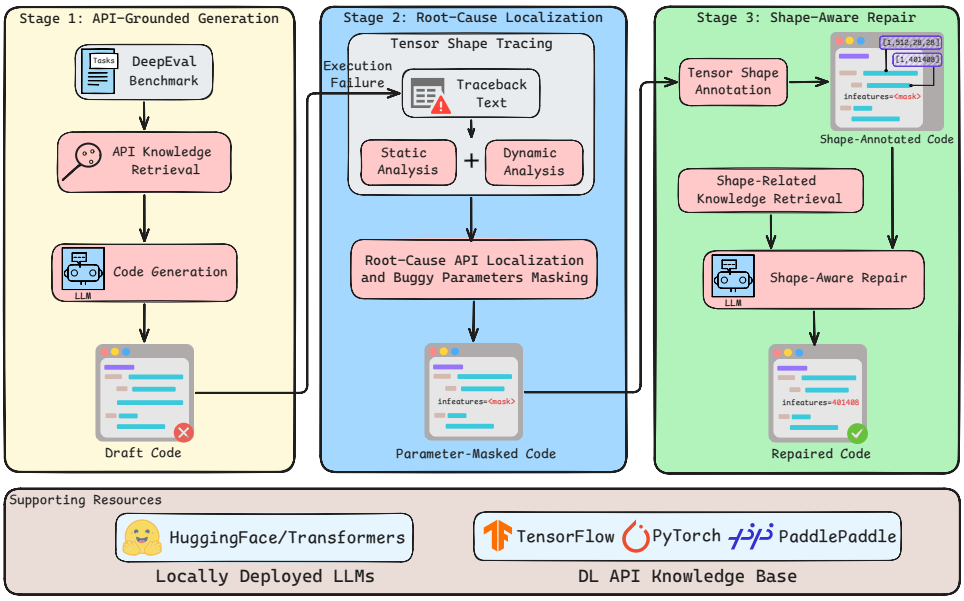


Fig. 1. DEEPGROUNDREPAIR tool workflow.

End-to-End Effectiveness. We use code executability, measured by Code Executability Rate (CER), as the primary effectiveness criterion because successful execution is a fundamental requirement for generated DL programs. As shown in Table I, DEEPGROUNDREPAIR reduces API-level syntactic issues, including API hallucinations and parameter misuses, by an average of 5.4 issues per 10 programs compared with the four baselines. As shown in Table II, it achieves the highest average CER across all three DL libraries, with an average relative improvement of 86.0% over the baselines. Compared with the two self-debugging baselines, it further improves Semantic Consistency Rate (SCR) by an average of 18.1%. SCR uses effective-line-count preservation as a proxy for semantic consistency, rather than directly establishing semantic equivalence.

Token Efficiency. Compared with the generation-only baselines (B1–B2), DEEPGROUNDREPAIR incurs 25.0%–63.0% higher token cost while improving CER by 54.0%–279.5%. More notably, compared with the self-debugging baselines (B3–B4), it reduces token cost by 18.5%–33.9% while improving CER by 13.7%–77.0%. These results demonstrate that DEEPGROUNDREPAIR achieves higher executability with lower token cost than traceback-guided self-debugging approaches. Further experimental results and detailed analyses are presented in the associated research study [9].

B. Representative Demonstration

Figure 2 illustrates the generation–localization–repair workflow using a representative PyTorch tensor-shape mismatch from the associated study [9].

(a) Draft Code with Bug. The draft program in Figure 2(a) defines a fully connected layer using `nn.Linear` with `in_features=512*7*7` and `out_features=1024`. However, the actual input tensor contains 401,408 features, rather than the configured 25,088. Consequently, execution fails at `self.fc1(x)` with a matrix-shape mismatch.

(b) Parameter-Masked Code. Although the traceback points to `self.fc1(x)`, the root cause lies in the upstream `nn.Linear` constructor. By combining traceback information with static and dynamic analysis, DEEPGROUNDREPAIR identifies `in_features` as the buggy parameter. It then masks this parameter with `<mask>`, as shown in Figure 2(b).

(c) Shape-Annotated Code. To provide shape evidence for repair, the tool annotates the parameter-masked code with runtime tensor shapes (Figure 2(c)). Specifically, the tensor shape changes from `(1,512,28,28)` to `(1,401408)` after flattening. These annotations help determine the correct input dimension of the fully connected layer.

(d) Repaired Code. Based on these shape annotations, DEEPGROUNDREPAIR replaces the masked `in_features` value with `401408`, as shown in Figure 2(d). The repaired program is then successfully passes the execution check, confirming that the tensor-shape mismatch has been resolved.

V. RELATED WORK

DL Program Bugs and Shape Repair. Prior studies have identified API misuse and tensor-shape mismatches as prevalent defects in DL programs, including LLM-generated code [1], [2], [7], [13]. Addressing these challenges requires not only correct API usage but also effective localization and repair of shape-related failures. DEEPGROUNDREPAIR integrates API-grounded generation, root-cause localization, and shape-aware repair to address these issues in a unified workflow.

Retrieval-Augmented Code Generation. Retrieval-augmented approaches improve code generation by incorporating external knowledge, particularly API documentation and search results [3]–[5], [14], [15].

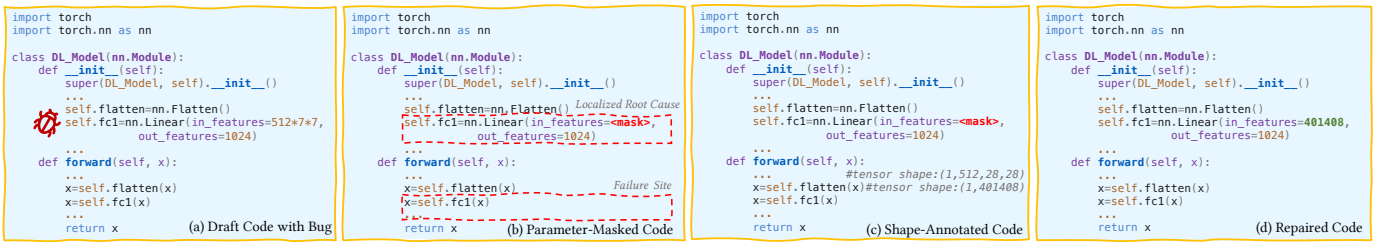


Fig. 2. Representative outputs of DEEPGROUNDREPAIR using Gemma-3-12B-it.

However, retrieved knowledge alone may not ensure valid API usage or resolve runtime failures caused by tensor-shape dependencies. DEEPGROUNDREPAIR addresses these limitations by verifying task-relevant APIs before generation and incorporating library-specific knowledge into repair.

Feedback-Guided Code Repair. Existing approaches leverage execution feedback to iteratively improve generated programs [12], [16]–[18]. However, traceback feedback may identify where a tensor-shape mismatch occurs without revealing the upstream API configuration responsible for it. DEEPGROUNDREPAIR addresses this gap through root-cause localization and explicit tensor-shape evidence to guide targeted repair. The underlying algorithms are detailed in the associated research study [9].

VI. CONCLUSION

We present the executable DEEPGROUNDREPAIR tool, which integrates API-grounded generation, root-cause localization, and shape-aware repair for LLM-generated DL programs. It supports TensorFlow, PyTorch, and PaddlePaddle and exposes intermediate artifacts for inspection and reproducibility. Evaluation results from the associated study show improved code executability and fewer API-level issues compared with generation and traceback-guided self-debugging baselines [9]. The demonstration illustrates how upstream shape evidence can guide a targeted, validated repair.

VII. ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China under Grants 62372021 and 62606045, the National Key R&D Program of China under Grant 2024YFB3311503, and the Fundamental Research Funds for Beijing Municipal Universities under Grant 312000546325001.

REFERENCES

[1] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *Proceedings of the 42nd International Conference on Software Engineering*, 2020, pp. 1110–1121.

[2] M. Wei, N. S. Harzevili, Y. Huang, Y. Jinqiu, J. Wang, and S. Wang, "Demystifying and detecting misuses of deep learning apis," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. tau Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[4] J. Chen, S. Chen, J. Cao, J. Shen, and S.-C. Cheung, "When LLMs meet API documentation: Can retrieval augmentation aid code generation just as it helps developers?" 2025. [Online]. Available: https://arxiv.org/abs/2503.15231

[5] N. Jain, R. Kwiatkowski, B. Ray, M. K. Ramanathan, and V. Kumar, "On mitigating code LLM hallucinations with API documentation," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 237–248.

[6] W. Gu, J. Chen, Y. Wang, T. Jiang, X. Li, M. Liu, X. Liu, Y. Ma, and Z. Zheng, "What to retrieve for effective retrieval-augmented code generation? An empirical study and beyond," Mar. 2025.

[7] X. Ma, X. Du, C. Li, J. Meng, X. Fang, W. Ding, and Z. Zheng, "How do large language models perform in deep learning code generation? an empirical study," *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: https://doi.org/10.1145/3816024

[8] Y. Deng, C. S. Xia, Z. Cao, M. Li, and L. Zhang, "Can LLMs implicitly learn numeric parameter constraints in data science apis?" in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[9] X. Ma, Z. Wang, X. Lu, X. Du, C. Li, J. Li, H. Li, and Z. Zheng, "DeepGroundRepair: From api-grounded generation to shape-aware repair for deep learning programs," 2026. [Online]. Available: https://arxiv.org/abs/2610.16897

[10] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, "Sparse, dense, and attentional representations for text retrieval," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 329–345, 2021.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 4, no. 1–2, pp. 1–174, 2009.

[12] X. Chen, M. Lin, N. Schaerli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html

[13] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019. [Online]. Available: https://arxiv.org/abs/1906.01388

[14] S. Zhou, U. Alon, F. F. Xu, Z. Jiang, and G. Neubig, "DocPrompting: Generating code by retrieving the docs," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://mlanthology.org/iclr/2023/zhou2023iclr-docprompting/

[15] K. Zhang, H. Zhang, G. Li, J. Li, Z. Li, and Z. Jin, "ToolCoder: Teach code generation models to use API search tools," 2023. [Online]. Available: https://arxiv.org/abs/2305.04032

[16] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: https://arxiv.org/abs/2207.01780

[17] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *International Conference on Learning Representations*, 2024.

[18] Y. Shi, S. Wang, C. Wan, M. Wang, and X. Gu, "From code to correctness: Closing the last mile of code generation with hierarchical debugging," 2024.